

AI TestFixer - Use Case Document Diagrams & Specification

Document Type:	Use Case Specification & Architecture Diagrams	Version:	1.0
Framework:	An Intelligent Agent-Based Self-Healing Framework for Selenium Test Automation	Date:	June 2026

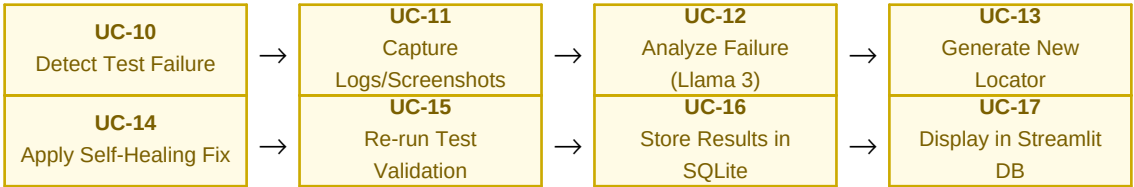
1. Introduction & Scope

AI TestFixer is an advanced intelligent self-healing framework designed to automatically detect Selenium test failures, capture relevant error contexts, analyze root causes using localized Large Language Models (Ollama Llama 3 AI), dynamically generate and apply repaired element locators, re-verify results, and archive all analytics into an SQLite backend visualized natively via a Streamlit multi-page interface. This document contextualizes and decomposes every critical functional dimension of the platform into precise use-case architectural representations.

2. System Pipeline Architecture (UC-10 to UC-17)

The automated execution lifecycle operates on a strict include-relationship pipeline. Once an original automation routine triggers a standard locator exception, control transitions through successive background worker agents. Each sequential system block perfectly encapsulates a singular functional requirement.

Figure 1: Core System Pipeline Include Relationship Chain



Pipeline Sequence Mapping:

- Core Execution Failure catches standard Selenium WebDriver locator faults dynamically (UC-10).
- Analysis and Fix Generation utilize localized LLM BeautifulSoup models without core script manipulation (UC-12, UC-14).
- Final output commits metrics instantly to the dashboard engine via an updated transaction loop (UC-16, UC-17).

3. QA Engineer Focus Area (UC-01, UC-02, UC-03)

The QA Engineer is primarily responsible for establishing parameters, initiating functional testing sequences, and vetting automated remediation pathways generated by localized models. They operate high-level views to ensure system behavior remains congruent with target requirements.

QA Engineer	UC-01 Execute Selenium Tests
	UC-02 View Dashboard Reports
	UC-03 Review AI Suggested Fixes

Figure 2: QA Engineer Interactivity Boundary

4. Developer Focus Area (UC-04, UC-05, UC-06)

Developers focus heavily on code stability, underlying script configuration, and approving proposed baseline locator modifications. Approved suggestions update permanent structures via the system learning agent.

Developer	UC-04 Manage Test Scripts
	UC-05 Review Locator Fixes
	UC-06 View Test Reports

Figure 3: Developer Action Context

5. DevOps / Admin Focus Area (UC-07, UC-08, UC-09)

The administration layer supervises framework health, executes periodic purge algorithms via maintenance sub-routines, archives transactional structures, and ensures standard dashboard operation.

DevOps / Admin	UC-07 Monitor Healing Analytics
	UC-08 Manage Database
	UC-09 View Dashboard

Figure 4: DevOps Administration Context Boundary

6. Actor Use-Case Mapping Matrix

The multidimensional intersection below highlights interactions spanning human operators and backend automated agents within the target software ecosystem.

Use Case ID & Title	QA Engineer	Developer	DevOps / Admin	System Background
UC-01 Execute Selenium Tests	✓	-	-	-
UC-02 / UC-09 View Dashboard/Reports	✓	✓	✓	-
UC-03 / UC-05 Review AI Recommendations	✓	✓	-	-
UC-04 Manage Test Scripts	-	✓	-	-
UC-07 Monitor Healing Analytics	-	-	✓	-
UC-08 Manage SQLite Database	-	-	✓	-
UC-10 to UC-15 Automated Core Pipeline	-	-	-	✓
UC-16 / UC-17 Data Storage & Display Sync	✓	✓	✓	✓

7. Platform Constraints & Specifications

- **Performance Threshold:** Streamlit views must fully populate cards within 3.0 seconds maximum execution runtime.
- **Resource Requirements:** Local Ollama instances running Llama 3 models mandate a minimum configuration of 8 GB RAM.
- **Storage Engine:** Embedded transactional data persists inside standard localized SQLite schemas without cloud provider endpoints.